TECHNICAL DESIGN · PHASE 1 · LOCAL-FIRST PATH

Phase 1: Qlik to Dremio SQL, Built and Validated Without Dremio

How the starting stage runs while Dremio access is not yet available: what changed from the architecture and workflow edition, what phase 1 requires, what is already built and what phase 1 still delivers.

Document	Technical design, phase 1
Companion to	Starting-Stage Proposal: Architecture and Workflow; Starting-Stage Technical Design
Prepared for	Engineering and technical reviewers
Date	October 2026
Status	Draft for review
Based on	Architecture v0.8, the implementation plan of 15 October 2026 and the code as built

Contents

1. Summary
2. What changed from the architecture and workflow edition
3. What phase 1 requires
4. What is built today
5. Phase 1 architecture
6. How a run works
7. Translation design
8. Validation without Dremio
9. Knowledge service adapter
10. Review loop
11. Use of AI
12. Environments and deployment
13. What phase 1 still delivers
14. Phase PD: connecting Dremio
15. Risks and mitigations
16. Open questions

1. Summary

The architecture and workflow edition assumed Dremio from the first day: the catalog crawl, running the generated SQL in Dremio and comparing results there. Dremio access is not available for the first phase. Rather than wait, phase 1 builds the whole Qlik-to-SQL path and proves it **locally**, and moves everything that needs a live Dremio into a separate phase, **PD: Dremio connection**.

Three things make this possible without weakening the result:

- **The output is still Dremio SQL.** The translator emits the Dremio dialect. For local runs the same SQL is transpiled to DuckDB; it is never rewritten by hand.
- **Qlik's own extracts are the evidence.** QVD files, their headers and the reload logs record what Qlik produced. The generated SQL is run locally and compared with them.
- **Dremio sits behind an adapter (DataPort).** Connecting it in PD swaps an adapter. The translation, agents, knowledge and review stay as they are.

The knowledge service follows the same pattern. It is an adapter (KGPort) with an open-source implementation today, Graphiti on Neo4j over an append-only Postgres fact log. The in-house engine (Brahmastra) plugs in later behind the same port.

Question	Short answer
What works today?	A Qlik application folder goes in. Dremio SQL per table, lineage, a validation level per table, review items and a migration pack come out. The test application reaches V4 locally on both tables.
What is different from the original plan?	No Dremio in phase 1, local validation (V0–V4, venue local), logical sources instead of resolved Dremio datasets, and an open-source knowledge engine behind an adapter (section 2).
What does phase 1 still need?	Real Qlik applications, a Google Cloud project for Vertex AI, export parsing for V5 and a staging deployment (section 13).
What happens when Dremio arrives?	Phase PD resolves logical sources to datasets and re-runs the same SQL in Dremio. Levels are re-earned there and recorded per venue (section 14).

2. What changed from the architecture and workflow edition

The architecture, the agent contracts and the validation ladder are unchanged. The table lists every point where phase 1 takes a different path, why, and when it returns to the original design.

Area	Architecture and workflow edition	Phase 1 path	Why	Back on the original path when
Data platform	Dremio from day one: crawl, run SQL, time travel	No Dremio. SQL is still written in the Dremio dialect and runs locally in DuckDB	No Dremio access in phase 1	A read-only service account arrives (Q-05, Q-06): phase PD
Source identity	Dremio datasets found by the catalog crawl	Logical sources from the scripts: connection + path, e.g. Dremio_Sales:sales.erp.orders_v2	Needs no Dremio; maps one-to-one later	PD resolves them, or a reviewer confirms a mapping
Source data	Dremio tables	QVD contents and source extracts (CSV) per logical source	Qlik's own outputs are independent evidence	PD

Area	Architecture and workflow edition	Phase 1 path	Why	Back on the original path when
Validation	V0–V5 in Dremio	V0–V4 locally, labelled venue local; V5 from exported reports next	Never claim a Dremio result that was not earned there	PD re-earns levels in venue dremio
Knowledge engine	Brahmastra evolved in place (ADR-12)	KGPort adapter: Postgres fact log + Graphiti 0.30 projection on Neo4j 5.26	Latest Brahmastra code not available (Q-02); Graphiti is open source (Apache-2.0)	Brahmastra is added as another KGPort implementation
Object storage	MinIO locally, GCS in cloud	SeaweedFS locally, GCS in cloud	MinIO images are no longer publicly available; code uses only the S3 API	Not needed
Orchestrator	Separate worker	Runs inside the API gateway process	One process is enough for one application at a time	Phase 1 (section 13)
Model gateway	Separate service	Library used by the agents; replay mode by default	No Vertex project yet (Q-04); tests stay offline	Phase 1 (section 13)
Sign-in	Keycloak single sign-on everywhere	API accepts Keycloak tokens; workbench login not yet	Local use first	Phase 1 (section 13)

3. What phase 1 requires

3.1 Inputs per Qlik application

Each application is one folder. Only the scripts are mandatory; every other input raises the level a table can reach. Missing inputs lower the level; they never stop a run.

Input	Used for	Required	Without it
Load scripts (.qvs, -prj folders, include files)	Everything: statements, tables, lineage, SQL	Yes	Nothing to translate
Reload log	Variable values at run time, rows per step (V3), errors	Strongly advised	Variables must be evaluated or answered by a reviewer; no V3 from the log
QVD files	Field lists for LOAD *, record counts (V3), contents (V4)	Strongly advised	Tables stop at V2
Source extracts, one CSV per logical source	Running the SQL locally for tables built from database sources	For tables read from databases	Those tables wait for PD (status waiting_for_dremio)
Exported reports	Figures match (V5)	Next in phase 1	No V5

```
<app>/
  script.qvs                      load script (and include files)
  reload.log                      Qlik reload log
  qvd/Orders.qvd                  QVDs the app reads or stores
  sources/sales/erp/orders_v2.csv one extract per logical source: sources/<space>/<folder>/<table>.csv
  private/                        real client material (git-ignored unless cleared)
```

3.2 Needed from the organisation

Item	Why	Status
2–3 real Qlik applications with reload logs and QVDs	Coverage and accuracy targets are measured on real scripts; synthetic fixtures alone overfit	Awaited
Google Cloud project with Vertex AI (Q-04)	AI fallback for translation and diagnosis, Gemini embeddings; recordings for replay in CI	Awaited; the system runs without it in replay mode
Read-only Dremio service account (Q-05, Q-06)	Phase PD only	Not needed for phase 1
Brahmastra code (Q-02)	Second KGPort implementation	Not needed for phase 1

3.3 Environments

Environment	Needs
Developer machine	Python 3.12 with uv, Node 22 for the workbench, Docker for Postgres 16, Neo4j 5.26 and SeaweedFS. Building the images needs several GB of free disk; the command line needs none of the services.
Staging (Google Cloud)	One project: GKE Autopilot, Cloud SQL Postgres 16 (private IP), Cloud Storage, Artifact Registry, Secret Manager, Vertex AI. All of it is described in Terraform.
Client install	The same Helm chart with client values: their cluster, their database, their storage and their model project. No data leaves the install.

4. What is built today

Status as of 15 October 2026. Each line was exercised by tests; the end-to-end lines also ran on the local stack through the API.

Component	What it does	Status
Qlik readers (adce-qlik)	Statement splitter with source spans, classifier, variables, reload logs, QVD headers, Lark grammar for LOAD, control flow (includes, SUB/CALL, loops, IF/SWITCH), BI IR builder with logical sources and QVD chains	Done
Translation (adce-reversesql)	Expressions with Qlik semantics made explicit, versioned CTEs per load step, Dremio dialect, checks	Done
Local validation	DuckDB + PyQvd venue, reconciler V0–V4	Done; test app at V4 on both tables
Knowledge service	KGPort; Postgres fact log + outbox; Graphiti projection on Neo4j; kg-mcp (FastMCP 3.4)	Done; integration-tested
Model gateway	Routes, privacy guard, memoization, record/replay, budgets, Claude on Vertex AI	Done; tested with a fake client
Agents and pipeline	1, 27, 28, 24, 25, 26, 19-lite, 20-lite; LangGraph qlik_pipeline with the Postgres checkpointer	Done
Review service	Ask, Queue, Gate, Seed; decisions become confirmed facts used by the next run	Done
Workbench	Overview, review queue, lineage (Next.js 16.4, React Flow 12.12)	Done; checked in a headless browser
API gateway	Uploads, runs, tables, facts, lineage, packs, review	Done
Command line	adce parse, adce translate, adce validate on a folder	Done

Component	What it does	Status
Deployment	Dockerfiles, one Helm chart (staging and client values), Terraform for Google Cloud, CI jobs	Helm lint and Terraform validate pass; images built locally
Not built yet	Export parsing and V5, measures from the front end (agent 29), Langfuse tracing, gateway and orchestrator as separate services, workbench login	Section 13

Verified end to end: uploading the test application through the API runs the full pipeline. Orders reaches V4 (local) with 35 of 35 comparisons and CustomerTotals reaches V4 (local) with 18 of 18. The run produces review items and a migration pack. The expected results are computed separately in plain Python, so the SQL is not checked against itself.

5. Phase 1 architecture

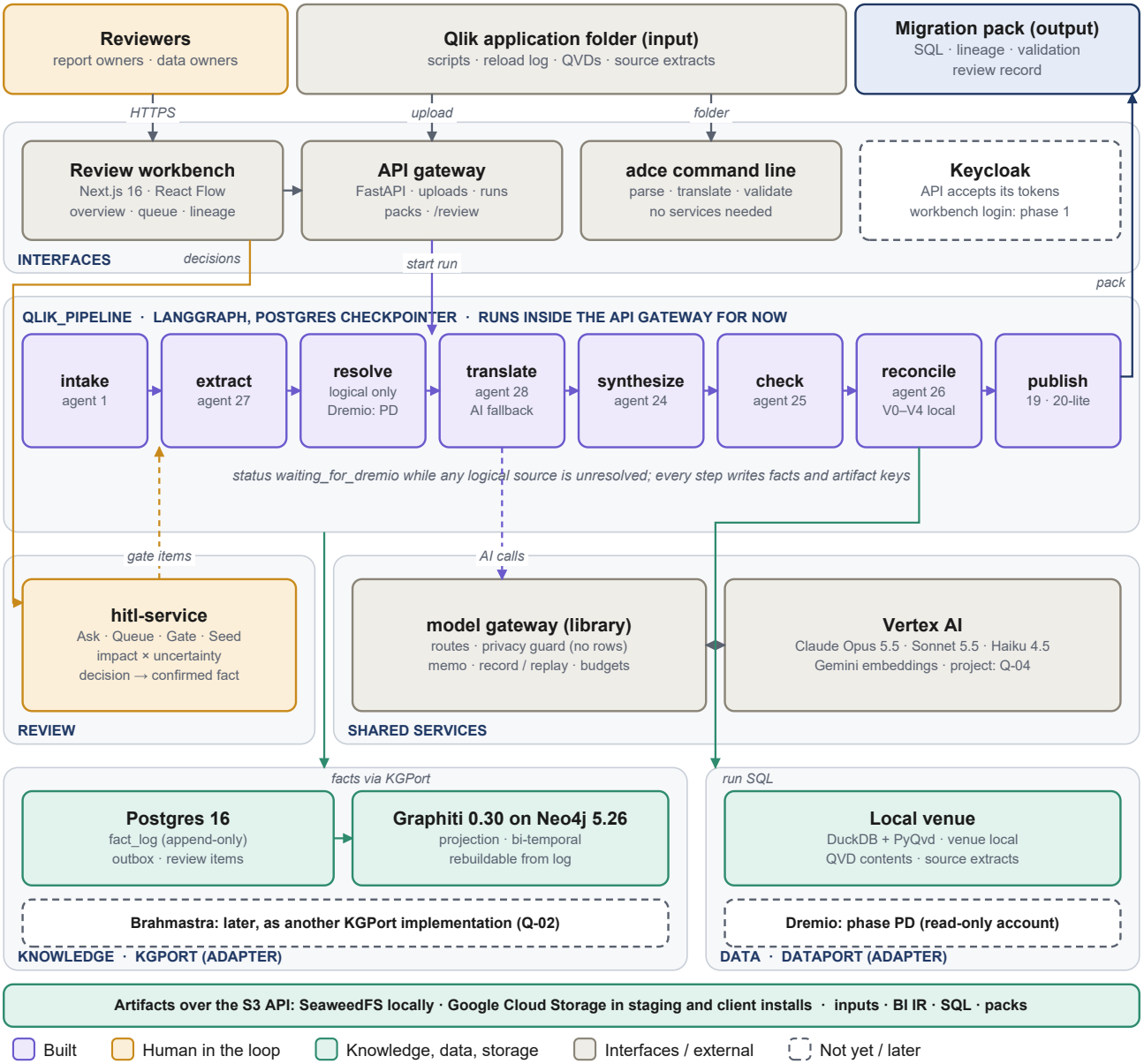


Figure 1. Phase 1 architecture. Solid boxes are built; dashed boxes come later in phase 1 or in phase PD. Knowledge and data are reached only through their adapters.

Every component has its final shape; phase 1 only changes what sits behind the two adapters. Agents write knowledge only through KGPort (the kg-mcp server exposes the same operations to external tools) and reach data only through DataPort. The translation is a pure function of the BI IR and the reviewer overrides, so any agent can recompute identical SQL and a pack regenerates identically.

6. How a run works

A run is one LangGraph execution of qlik_pipeline for one application. State is checkpointed in Postgres, so a run resumes after a restart. Steps exchange artifact keys, not payloads.

Step	Agent	What happens	Records
intake	1 Intake router	Classifies every file: script, include, reload log, QVD, source extract, export	file facts, artifact keys
extract	27 BI extractor	Splits and classifies statements, runs control flow, applies confirmed variable answers, builds the BI IR	BI IR, coverage, logical sources, issues
resolve	4-lite (logical)	Keeps logical sources; applies reviewer-confirmed mappings; marks the rest pending	scope links, resolution status
translate	28 Script translator	Rules first; reviewer SQL as overrides; AI fallback for statements the rules cannot translate	statement translations with tier
synthesize	24 Synthesizer	Composes versioned CTEs per table in the Dremio dialect	SQL per table
check	25 Query checker	Syntax, transpiles, known fields, determinism, inputs to resolve, AI-assisted marker	check results, V1
reconcile	26 Reconciler	Runs the SQL locally and compares with reload log and QVDs	level per table, comparisons
publish	19-lite, 20-lite	Lineage graph and migration pack: sql/*.sql, lineage.json, validation.md, review.md, manifest.json	pack key
summarize	orchestrator	Run status: complete, needs review, or waiting_for_dremio	run summary

Review items are generated from the facts after each run. A reviewer's decision is stored as a confirmed fact, and the next run consumes it: reviewer SQL replaces a translation, a variable answer seeds the interpreter, a confirmed mapping resolves a source.

7. Translation design

Translation runs in two layers. Expressions go from Qlik's ExprAST to sqlglot syntax trees; load steps go to versioned CTEs, one per step, so the SQL mirrors the order Qlik ran the script in. Qlik behaviour that SQL does not share is written out explicitly instead of being assumed.

Qlik construct	SQL produced	Why
a & b	NULL-safe concatenation	Qlik treats NULL as empty text in &
x / y	x / NULLIF(y, 0)	Qlik returns NULL on division by zero
Comparisons and If()	CASE WHEN … ; truth values -1 / 0 where a value is needed	Qlik's true is -1
LIKE with * and ?	ILIKE with % and _	Qlik's LIKE is case-insensitive
Date(), Num(), Text()	The value passes through unchanged	Formatting is presentation, not data

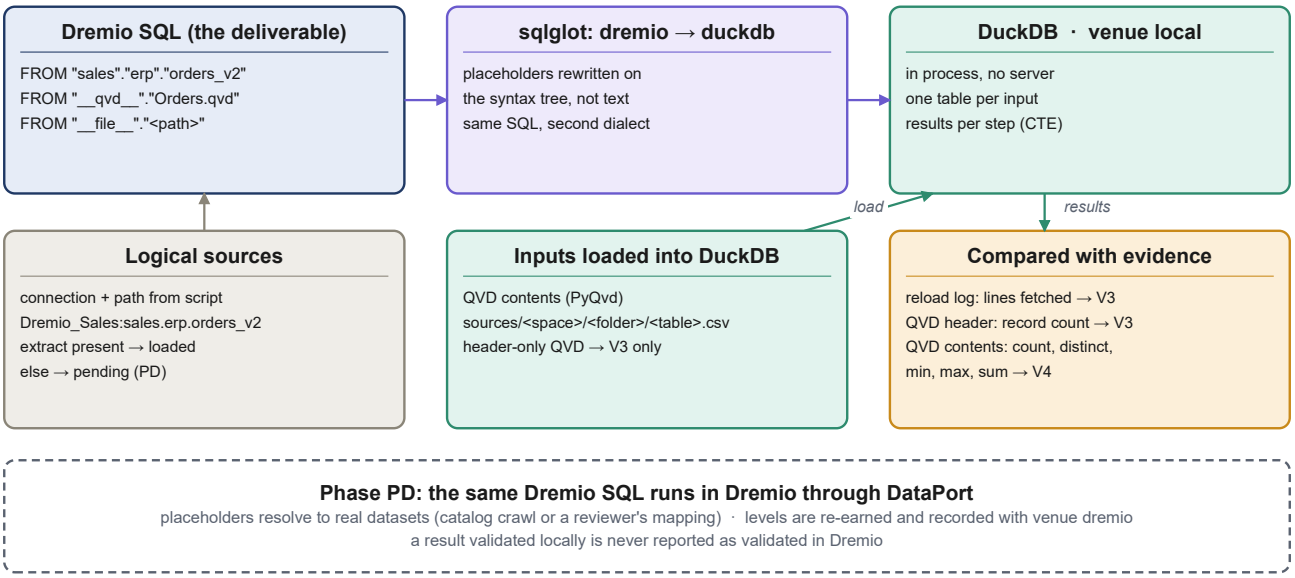
Qlik construct	SQL produced	Why
ApplyMap('M', k, d)	LEFT JOIN to MIN(value) per key; CASE WHEN map_key IS NULL THEN d	First match per key; key uniqueness is checked locally
Concat()	LISTAGG	Dremio's aggregate for text
CONCATENATE / automatic concatenation	UNION ALL into a new table version	Same field set or explicit prefix
JOIN (outer, left, right, inner)	Join on shared field names; outer joins COALESCE the keys	Qlik joins on all common fields
KEEP	EXISTS semi-join on both tables	Filters without merging
RESIDENT, DROP, STORE	Reads the table version current at that point; STORE snapshots it	Order matters in Qlik scripts
QVD and file inputs	"__qvd__"."<path>" and "__file__"."<path>"	Resolved to datasets later; local runs map them to loaded inputs

Example: the Orders table of the test application. Comments point back to the script lines each CTE came from; reviewers read the SQL next to the script.

```
t_orders_1 AS (
  -- load: statements 6 (script.qvs:17-21)
  SELECT
    "src"."OrderID" AS "OrderID",
    "src"."Amount" AS "Amount",
    CASE WHEN "src"."Amount" > 1000 THEN 'Large' ELSE 'Small' END AS "OrderSize",
    CASE WHEN "m1"."map_key" IS NULL THEN 'Unknown' ELSE "m1"."map_value" END AS "RegionName"
  FROM t_orders_sql_1 AS src
  LEFT JOIN (
    SELECT "Region" AS "map_key", MIN("RegionName") AS "map_value"
    FROM t_regionmap_1 GROUP BY "Region"
  ) AS m1 ON "m1"."map_key" = "src"."Region"
  WHERE "src"."Status" <> 'Cancelled'
),
```

A statement the rules cannot translate is never dropped. It is marked untranslated with its tier and span, sent to the AI route when a Vertex project is configured, and otherwise becomes a review item. AI-produced SQL is re-checked by the same checker and carries an AI-assisted marker.

8. Validation without Dremio



Each table carries its highest level and the venue that earned it, e.g. Orders: V4 (local), 35/35 comparisons ok.

Figure 2. The local validation path. The SQL that is shipped is the SQL that is run; only the dialect and the table names change, both on the syntax tree.

Level	Meaning	Evidence	Phase 1	Phase PD
V0	Parsed	Script text	Yes	Unchanged
V1	Checked	Rules, known fields, Dremio syntax	Yes	Plus Dremio catalog
V2	Executes	Runs without error	Local (DuckDB)	In Dremio
V3	Row counts	Reload log lines fetched per step; QVD header record counts	Local	In Dremio, at the reload time where Iceberg allows
V4	Data matches	Count, distinct, min, max, sum per field against QVD contents	Local	In Dremio
V5	Figures match	Exported reports	Next (section 13)	In Dremio
V6	Engine match	Qlik server	Later stage	Later stage

Two choices keep the levels honest. For V3, Qlik's "lines fetched" is matched against both the rows a step read and the rows it produced, and the report records which one matched. A QVD with a header but no readable contents still supports V3 and stops there.

9. Knowledge service adapter

Agents depend on KGPort, never on a graph engine. The phase 1 implementation keeps the record in Postgres and the queryable graph in Neo4j:

- **Fact log (record).** Every fact is appended to fact_log with its agent, evidence, trust level and run. A database trigger rejects updates and deletes. An outbox row is written in the same transaction.
- **Projection (derived).** The outbox is projected into Graphiti 0.30 on Neo4j 5.26 as entity nodes, bi-temporal edges and episodes. It is written directly, without Graphiti's LLM deduplication, so writes are deterministic. The graph can be rebuilt from the log at any time.
- **Policy.** A fact without an agent, or of a kind the agent may not write, is rejected before it reaches the log. Reviewer decisions are facts with origin reviewer and trust confirmed.

- **Embeddings.** Gemini embeddings on Vertex AI when a project is configured; a deterministic hash embedder offline. Graphiti telemetry is off.

Brahmastra (Q-02) becomes a second KGPort implementation, and the same test suite decides when it replaces or joins the Graphiti projection.

10. Review loop

Mode	When it is raised	What a decision does
Ask	A question only a person can answer, e.g. the value of a variable set outside the script	Stored as a variable_value fact; seeds the interpreter on the next run
Queue	Work that can wait, e.g. an untranslated statement	Reviewer SQL is stored as a translation_override and used instead of the rules
Gate	A result that must not ship unconfirmed, e.g. AI-assisted SQL	Confirm or reject on report_sql; rejected SQL is not published
Seed	Knowledge that unblocks a run, e.g. which Dremio dataset a logical source is	Stored as resolves_to; the source counts as resolved

Items are ranked by impact (how many tables and figures depend on the item) times uncertainty (how weak the evidence is). The workbench shows the queue, the SQL next to the script, and the lineage.

11. Use of AI

The rule-based path translates and validates the test application with no model calls. AI is used only where rules run out, always through the model gateway:

Route	Model (Vertex AI)	Used for
translate_statement, translate_expression	Claude Sonnet 5.5	Statements and expressions the rules cannot translate
repair_sql, diagnose_mismatch	Claude Opus 5.5	SQL that fails a check; explaining a validation mismatch
parse_export	Claude Sonnet 5.5	Reading exported reports for V5
classify_file	Claude Haiku 4.5	Files the intake rules cannot classify

- **Privacy guard:** prompts carry script text, field names and statistics, never data rows.
- **Record and replay:** every response can be recorded; CI and offline runs replay recordings, so results are reproducible and cost nothing.
- **Memoization and budgets:** identical requests are answered once; each run has a spending limit.
- **Re-checked by code:** AI output goes through the same checker and validation ladder as rule output and is marked AI-assisted.

12. Environments and deployment

Piece	Local	Staging and client installs
Postgres 16	Docker Compose	Cloud SQL (private IP, point-in-time recovery) or the client's
Neo4j 5.26	Docker Compose	In the cluster (Helm) or a managed instance
Object storage	SeaweedFS (S3 API)	Google Cloud Storage through its S3 API (HMAC key)

Piece	Local	Staging and client installs
Services	uv run, one process each	One Python image for all services, one workbench image; one Helm chart
Migrations	alembic upgrade head	Helm pre-install / pre-upgrade hook
Identity	Optional Keycloak profile	Keycloak; workload identity to Vertex AI
Infrastructure	None	Terraform: VPC with private service access, GKE Autopilot, Artifact Registry, Cloud SQL, Cloud Storage, Secret Manager, workload service account with Vertex AI user

CI runs lint, type checks and the tests (with Postgres and Neo4j as service containers), builds the workbench, lints the Helm chart, validates Terraform and builds both images.

13. What phase 1 still delivers

Work package	Scope	Done when
1.1 Real applications	Onboard 2–3 real Qlik applications; add a fixture and a test for every construct they use that is not yet supported	Every application parsed; coverage reported; nothing dropped silently
1.2 Translation on real scripts	Close the rule gaps the real scripts show; AI fallback on Vertex with recordings	≥ 90% of statements translated; ≥ 80% of tables at V3 (local)
1.3 Measures	Front-end expressions and measures (agent 29); grain and fan-out checks	Measures translated with grain checked
1.4 Figures (V5)	Parse exported reports (agents 21, 22); compare figures	≥ 3 real figures match exported reports
1.5 Platform	Langfuse tracing; model gateway as an HTTP service; orchestrator as a separate worker; Keycloak login in the workbench	Same run traced end to end with sign-in
1.6 Gold set and evaluation	Expected outputs for the real applications; evaluation in CI with replayed model calls	Regressions fail CI
1.7 Staging	Terraform apply, images in Artifact Registry, Helm install	The same run gives the same pack in staging

> **Phase 1 is complete when a migration pack for a real application is accepted by its report owner, the pack regenerates identically, and the same build runs in staging.**

14. Phase PD: connecting Dremio

PD can start at any point in phase 1; it needs only a read-only Dremio service account. It adds:

Step	What it does
DataPort for Dremio	Arrow Flight SQL via ADBC for queries; REST catalog API v3 for metadata
Catalog crawl (agents 2, 5)	Datasets, view SQL, wikis and tags into the knowledge service
Source resolution (4-lite)	Logical sources → Dremio datasets, with reviewer confirmation where the match is not exact
Grain and copies (9-lite, 30-lite)	Keys and grain measured in Dremio; near-duplicate datasets clustered

Step	What it does
Re-validation	The same SQL runs in Dremio; V2–V5 re-earned with venue dremio, at the reload time where Iceberg time travel allows

Nothing upstream changes: the translator already writes the Dremio dialect, and placeholders are replaced on the syntax tree, as in local runs.

15. Risks and mitigations

Risk	Effect	Mitigation
DuckDB and Dremio differ on a function or type	A local V4 that does not hold in Dremio	Dremio dialect is primary; local results are labelled; PD re-validates every table
Source extracts differ from what Qlik read	False mismatches or false matches	Extracts are taken at the reload time; QVD contents are preferred evidence where they exist
Synthetic fixtures overfit the rules	Coverage looks better than it is	Targets are measured on real applications only (1.1, 1.2)
Graphiti changes quickly	Upgrades break the projection	Pinned version; the projection is rebuildable from the fact log; KGPort hides it
Dremio access arrives late	V2–V5 stay local	All non-Dremio work proceeds; PD is separate and can start any time
No Vertex project	Long-tail statements wait for review	Review queue handles them; replay keeps tests offline
Disk space on developer machines	Image builds fail	Builds also run in CI

16. Open questions

Question	Affects	Assumption until answered
Real Qlik applications and their QVDs	1.1, 1.2, every target	Synthetic fixtures only
Google Cloud project for Vertex AI (Q-04)	AI fallback, embeddings, staging	Replay mode; hash embeddings
Dremio edition, access, permissions (Q-05, Q-06)	Phase PD	Local venue
Brahmastra code (Q-02)	Second KGPort implementation	Graphiti on Neo4j
Qlik data connections (Q-35)	Source resolution	Logical sources, reviewer-confirmed mappings
Neo4j edition for client installs (Q-19)	Licensing	Community edition for development

The full list with status is kept in docs/open-questions.md.